HOW MUCH HUMAN WORK IS BEHIND AI-ASSISTED SOFTWARE?
LABOUR BOUNDS AND ESTIMATION FROM WORK TELEMETRY

Abstract. Artificial intelligence (AI) can change the relationship between development events, such as Git commits, and the human work behind them. We measure that work in person-hours. To relate labour to delivery, we also measure elapsed time to first acceptance and assess how much of the accepted requirement is still satisfied a specified time after first acceptance. For a task accepted by the deadline, multiply its reference score by this fulfilment fraction to obtain a weighted score. Tasks that miss the deadline receive a weighted score of zero. We examine how measured labour totals for time intervals constrain the labour possible within other periods, and how sampled labour measurements can correct predictions based on development data, such as commit histories.

Given observed labour totals or justified bounds on those totals, each person's available time limits how much labour can fall within a specified period. Dividing time at the start and end of every observed interval and every period whose labour total is sought gives a finite linear programme with exactly the same possible labour totals. This remains true when observations overlap or several tasks share a person's time. Enforcing each person's capacity across tasks can rule out totals included when separately calculated bounds are added. Whenever at least one allocation satisfies the observations and capacity limits, every total from the minimum to the maximum is possible. Each total can also be achieved by a schedule in which each person does at most one activity at a time, provided work may start and stop at arbitrary times.

For estimation, we list the time intervals for each person whose labour is included and select intervals at random, giving each a known positive probability of selection. In each selected interval, we measure labour and subtract its prediction. Adding each difference divided by its selection probability to the total predicted for all listed intervals corrects that total. Predictions are calculated before selection and remain unchanged across the possible samples. With accurate labour measurements, the corrected estimate averages to true total labour over repeated samples. Predictions can increase or decrease its variance. The results follow from proofs within the stated allocation and sampling models. Lean checks supporting results about interval integrals and the schedules used in the proofs, while exact finite calculations check numerical examples. Empirical validation of the measurements is the next step towards using them to compare AI-assisted work.

1. What the observations determine

Suppose one person spends three hours working on a task during a four-hour interval. How much of that work occurred in the first two hours? Either half can contain at most two person-hours, so at least one person-hour must fall in the first half. The person could work one hour in the first half and two in the second, or two in the first and one in the second. Both limits are possible, giving a first-half labour range of one to two person-hours. Commit times leave a further ambiguity because the same sequence of commits can arise from different amounts of human activity.

Labour is human time allocated to the work included in the study, measured in person-hours. In software work assisted by artificial intelligence (AI), it includes unsuccessful attempts and repair after acceptance as well as work leading to acceptance. To compare this labour with delivery, we also measure elapsed time to first acceptance and assess the accepted requirement a specified time later. For tasks accepted by the deadline, multiply the reference score by the fraction of that requirement still satisfied. This weighted score is zero for tasks that miss the acceptance deadline.

Date: 5 November 2026.

Key words and phrases. software engineering, partial identification, labour bounds, activity sampling, model-assisted estimation, AI assistance.

When several tasks share a person's time, their allocation fractions must sum to at most one across all tasks and types of work. This capacity constraint is the limit used in the opening example. Provided the observations and this constraint allow at least one allocation, our linear programme finds the smallest and largest possible labour totals.[1] Every total between them is also possible. Narrowing the range therefore requires further observations or justified restrictions on how people divided their time.

When exact labour totals are unavailable, we can estimate them by measuring labour in selected time intervals for individual people. Divide each person's period of work being studied into non-overlapping intervals and list them all. A probability sampling design selects from this list at random, giving every interval a known positive chance of selection. Predictions calculated for all listed intervals before selection give an initial total. We correct it using the selected intervals' differences between measured and predicted labour, each divided by that interval's selection probability. With accurate measurements of the sampled allocations, the corrected estimate averages to the true total over repeated samples. Predictions improve precision only when estimating their total error has lower sampling variance than estimating labour without them.[2] Commit timestamps and code-change features can therefore aid labour estimation even when commits alone leave its exact value unresolved.

How do these two approaches describe uncertainty about total labour? The labour bounds identify exactly which totals are compatible with the observations and the rules for allocating each person's time. For the sample-based estimate, sampling uncertainty describes how the estimate would vary if the sampling procedure were repeated.

Interpreting a score derived from commit histories as person-hours requires calibration against labour measured separately from those histories. Standard Coder illustrates this requirement. Wright and Ziegler's method [25] infers coding activity from commit timing, then predicts those activity labels from code-change features. Its Standard Coding Hours can provide a common reference index. Calibration is needed because human activity and code changes can correspond to different amounts of labour, especially under AI assistance. This measurement question sits within broader accounts of productivity. Forsgren et al. distinguish activity and efficiency within their developer-productivity framework [12], while Mozannar et al. examine the time costs of interacting with AI-assisted programming tools [20].

Estimating AI's effect on labour or task delivery requires a causal study design that supports a comparison between outcomes with AI assistance and those that would arise without it. The chosen outcome determines what such a comparison establishes. Peng et al. examine completion time among completers of a programming exercise [21]. Becker et al. study experienced maintainers in familiar repositories, comparing log completion times after adjustment for forecast times [4]. Cui et al. study weekly completed pull requests in organisational experiments [8], using allocation to study groups as an instrument for adoption in their June 2025 manuscript [7, Section 4.1 and Table 3]. These studies use different populations and working conditions as well as different outcomes. Their estimates therefore answer different questions about AI-assisted work. A subsequent report from Model Evaluation and Threat Research describes selection and concurrent-agent measurement difficulties, supporting the need to measure human labour separately from elapsed time [5]. The calculations here use no data from these studies.

Sections 2 and 3 explain why commits alone can leave person-hours unresolved, then define labour, acceptance time and weighted scores. Section 4 derives the full range of labour

[1] A linear programme minimises or maximises a weighted sum subject to linear equalities and inequalities [6, Section 4.3]. Here the unknowns are hours allocated within specified time intervals. The inequalities limit those hours using the observations and each person's available time.

[2] Sampling variance measures how widely estimates vary across the possible samples, as their probability-weighted mean squared distance from the average estimate. An unbiased estimator has an average equal to the true total. A particular sample can still give an estimate above or below that total.

totals consistent with interval observations. Section 5 turns to sampling and uses measured labour to correct predictions calculated before the sample is selected. Section 6 sets out the measurement checks needed before comparing AI tools and working practices. The proofs apply standard optimisation and sampling results to the stated rules for allocating people's time. Appendix A gives a shortest-path solution for one person's work of one type on one task.

2. Why commit histories can leave labour unresolved

Why can the same commit distribution correspond to different amounts of activity? Commits in the model below depend on how often activity occurs and how often an active minute produces a commit. More frequent activity can be offset by a lower chance of a commit during each active minute.

To express this relationship, let $Z_t \in \{0,1\}$ be the hidden activity state in minute t, and let $C_t \in \{0,1\}$ indicate whether a commit occurs in that minute. A fitted event model specifies how activity changes from minute to minute and how commits depend on that activity. Interpreting $Z_t = 1$ as "coding" requires validation against labour measured separately from the commit history. The following example shows how these two probabilities can vary while the commit distribution stays the same. A Bernoulli variable takes the value one with its stated probability and zero otherwise.[3]

Proposition 2.1 (Failure of unrestricted identification)**.** *Suppose the states Z_t are independent Bernoulli variables with parameter $\pi \in (0,1)$, and, independently across minutes and of the states, B_t are Bernoulli variables with parameter $c \in (0,1)$. Define $C_t = Z_t B_t$. The joint distribution of all commit observations depends on (π, c) only through πc. In particular, $(\pi, c) = (1/2, 1/5)$ and $(1/4, 2/5)$ give identical commit distributions but different expected active times.*

Proof. Each C_t is Bernoulli with parameter $\theta = \pi c$, and the C_t are independent. For every $n \geq 1$ and every binary string $(e_1, \ldots, e_n)$,

$$\mathbb{P}(C_1 = e_1, \ldots, C_n = e_n) = \theta^{\sum_t e_t}(1-\theta)^{n-\sum_t e_t}. \tag{1}$$

Both parameter pairs give $\theta = 1/10$. In contrast,

$$\mathbb{E}\left[\sum_{t=1}^{n} Z_t\right] = n\pi, \tag{2}$$

which is $n/2$ minutes in the first model and $n/4$ in the second. Thus identical commit distributions can correspond to different expected numbers of active minutes. □

In Equation (1), e_t specifies whether minute t contains a commit, and $\sum_t e_t$ counts those minutes across the specified sequence. Equation (2) instead gives the expected number of hidden active minutes. Both models assign the same probability to every commit sequence while giving different expected active times. This is why the commit distribution alone cannot distinguish the two models.

This example is a special case of a two-state hidden Markov model with independent activity states, in which the next activity state has the same distribution for either current state. It shows that this model family allows different activity levels to produce the same commit distribution. Generic-identifiability results for finite-state hidden Markov models concern parameters outside exceptional sets and permit hidden-state relabelling

[3]In probability notation, $\mathbb{P}(A)$ is the probability of event A, and $\mathbb{E}[X]$ is the probability-weighted average of a random quantity X. The symbol $\sum_{t=1}^{n}$ means addition over minutes 1 to n. The notation $Z_t \in \{0,1\}$ means that Z_t is either zero or one. The interval $(0,1)$ contains values strictly between zero and one. Here π denotes an activity probability.

[2, Section 6.1].[4] To identify the parameters, state which model restrictions or additional measurements distinguish the possible models. Restrictions on the activity model, informative code-change features or separately obtained measurements of activity may supply that distinction.

Even after the activity model is identified, interpreting its states as human labour requires validation against person-hours measured separately from the commit history used to fit it. Under AI assistance, commits can continue while human attention is elsewhere, so the relationship between activity and commits may change. Measured interval totals instead constrain how much labour occurred during specified periods. Using those measurements requires a definition of which work belongs to each task and when that work is counted.

3. Counting labour and assessing task outcomes

3.1. **Defining tasks and allocating human time.** Identify each task by the change requested and the criteria for accepting it. A GitHub issue, for example, can describe the requested behaviour and the checks a reviewer will use to accept the implementation. Its commits and pull requests can change while the issue continues to identify the same requested work. The study protocol states which tasks are eligible for inclusion. Begin tracking task i at time s_i, when its requirement, acceptance criteria and eligibility under those rules have been documented. Keep the requirement version used at that time, for example by saving the issue description when tracking begins.

Requirements can later change, and tasks can be split or merged. Define how these changes will enter the analysis before inspecting labour totals, acceptance times or weighted scores. In the GitHub example, dated issue edits show which requirement or acceptance criterion changed. Links between issues identify the original task and any components or merged tasks. Decide whether accepted components receive their own scores or contribute only through the completed parent, counting each contribution once. If one component is accepted before the deadline and another afterwards, counting components recognises the first in time, while counting the parent waits for both. Regrouping can therefore change the weighted total even when the component reference scores, defined below, sum to the parent's score.

AI access is part of the working conditions being compared. The study plan must specify which tools and model versions each group may use, which tasks they may assist and which autonomous actions are permitted. It must also identify the required human review and release approval, including who is responsible for those checks. Record actual tool use separately from these permissions. For example, a task diary can identify the AI tool used and the work it assisted. Access and use can differ because difficult tasks may prompt more use, while people may decline an available tool. Comparing groups by use alone can therefore mix the effect of access with differences in task difficulty.

How should human time be counted when several AI workflows run at once? Count the time each person spends on the work and calculate automated execution time separately. Let $a_{ijr}(t)$ be the fraction of person j's time spent on task i doing work of type r, such as implementation or review, at calendar time t. Each person's combined allocation is at most one. Integrating an allocation fraction over time gives the person's labour on that task and type of work. Adding these contributions gives the task's total labour. With finite sets of tasks, people and types of work, and measurable allocation functions,[5] labour through task

[4] In a hidden Markov model, the next hidden state depends on the current state, and observations are conditionally independent given the state sequence, each depending on its corresponding state. Generic identifiability means that the observed distribution determines the parameters, up to state relabelling, outside an exceptional set of parameter values. Relabelling changes the names of states and their associated parameters together. The example above lies in the independent-state subfamily.

[5] The integral $\int_s^e a(t)\,dt$ accumulates the allocation fraction over elapsed time. A constant allocation a over $e - s$ hours contributes $a(e - s)$ person-hours. These are Lebesgue integrals with respect to time. Measurability ensures that sets defined by allocation thresholds have a well-defined duration. The phrase

age $u \geq 0$, measured from the start of tracking, is

$$a_{ijr}(t) \geq 0, \qquad \sum_{i,r} a_{ijr}(t) \leq 1, \qquad H_i(u) = \sum_{j,r} \int_{s_i}^{s_i+u} a_{ijr}(t)\, dt. \tag{3}$$

With time in hours, $H_i(u)$ has units of person-hours. Two people working for an hour can therefore contribute two person-hours. The allocation constraint limits each person's combined contribution during that hour, including when several sessions run concurrently.

A person may switch between prompting one workflow and reviewing or integrating another's results while both workflows continue to run. The allocation functions represent the fraction of the person's time spent on each activity, including supervision. Dividing that time between tasks keeps the combined allocation within the person's available time, so each period of human work is counted once.

For example, consider one person using two AI workflows during a one-hour interval. The person prepares workflow A's prompt during minutes 0–10 and submits it at minute 10. Workflow A starts then and finishes at minute 60. The person prepares and submits workflow B's prompt during minutes 10–20. Workflow B executes from minute 20 to minute 40, after which the person reviews its result until minute 60. Assume no other human work on these tasks during the interval. Human labour totals forty minutes, or 2/3 of a person-hour. Adding the two execution durations gives seventy minutes, or 7/6 workflow-hours. Figure 1 shows how both totals arise.

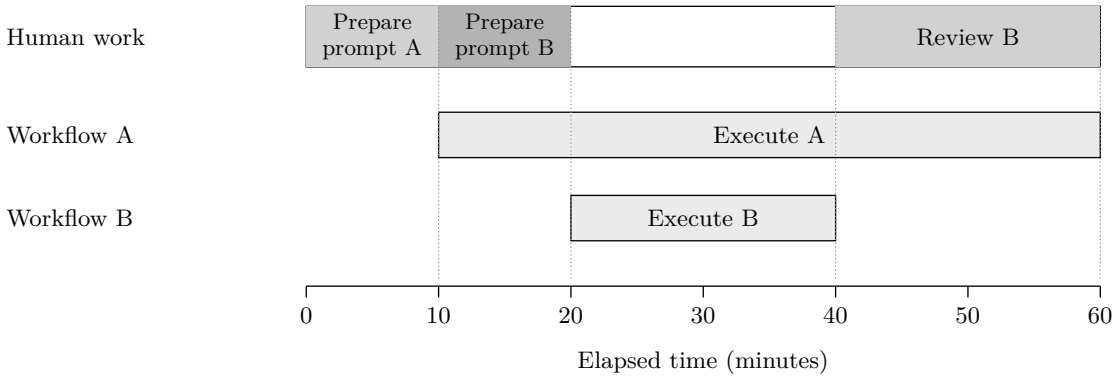


FIGURE 1. **Human work and automated execution occupy different periods.** In this illustrative one-hour interval, one person prepares prompt A during minutes 0–10 and prompt B during minutes 10–20, submitting each when preparation ends. Workflow A then executes until minute 60. Workflow B finishes at minute 40, and its result is reviewed during minutes 40–60. The shaded human-work periods total forty minutes, or 2/3 of a person-hour. The two execution periods total seventy minutes, or 7/6 workflow-hours, including twenty minutes when both workflows run. During minutes 20–40, the person does no work on either task.

To apply this labour account, decide which activities count and how shared work is assigned to tasks. Include the work from specification and implementation through review, testing and repair, and state whether attended waiting counts. Shared review and platform work need an attribution rule so their person-hours can enter Equation (3). List measured person-hours that cannot yet be assigned to a task separately from task totals. An unknown task assignment leaves the work's attribution unresolved even when its duration is known.

"almost everywhere" used below permits exceptions on a set of total duration zero, such as individual switching instants. Such exceptions leave these integrals unchanged.

Count automated execution separately from human labour, including when a person attends to a running workflow. In the example, a workflow-hour is one hour of elapsed execution for one workflow. Add the durations of the individual runs within the observation period, so overlapping runs each contribute their own duration. State which start and stop events delimit a run. CPU usage, token use and charges require separate measurements with their own units. Combining human time with computing resources requires a stated conversion into costs. The combined measure then describes cost rather than labour.

Elapsed time to acceptance includes periods when the task is queued or its workflows run without human attention. Let T_i be elapsed time from the start of tracking to first acceptance. Starting at s_i counts queueing and any reasoning or specification between s_i and the first commit. If the task is permanently abandoned before acceptance, put $T_i = \infty$. If the study loses track of an unfinished task, its eventual acceptance time is unknown and is treated as censored. Acceptance and deployment may occur at different times. Keep their timestamps separately, for example an acceptance confirmation in the task issue and a deployment timestamp in a release log.

3.2. **Scoring tasks and assessing requirement fulfilment.** Assign each task a non-negative reference score S_i^0 under a common scoring rule. The score may count tasks or functional units, or use Standard Coding Hours as weights. Keep each value with the scoring-rule version and the inputs used to calculate it. For example, a task's scoring entry could identify its GitHub issue and requirement version, then give the assigned reference score. The superscript 0 denotes the common reference. Keep each task's reference score unchanged across the comparisons, using the same scoring rule and weights for both groups.

The information used to score a task determines whether AI can change the reference score itself. For a causal comparison of fixed requirements, prefer the requirement version saved when tracking begins. A score based on final code depends on the implementation, which AI may change, so reporting such a score requires making that dependence explicit. Splitting or squashing a code change can also alter a nonlinear score. To keep the reference consistent, score the same task information once or use a requirement-scoring rule whose component scores sum to the total. Count the delivered requirement once across retries, failed attempts and reverts.

A score expressed in person-hours needs calibration against labour measured separately from the commit histories used by the predictor. Specify whose labour is included, which types of work count and which quality requirements apply. Also state the last task age included in the labour total. A predictor trained on activity states inferred from commits learns to predict those states. Interpreting its predictions as person-hours requires comparison with separately measured human time under these inclusion rules and the same task-age limit.

How does the later requirement assessment enter the calculation? For a task accepted by the deadline, multiply its reference score by the fraction of its accepted requirement still satisfied at that assessment. A task that misses the deadline contributes a weighted score of zero. Choose an acceptance deadline at task age $\tau > 0$ and an assessment delay $\Delta \geq 0$ measured from first acceptance. This later check is the follow-up assessment. Specify how reviewers will judge partial fulfilment of the accepted requirement. For accepted tasks, let $R_i(\Delta) \in [0, 1]$ be the fraction still satisfied at time $s_i + T_i + \Delta$, judged using these criteria. The task's weighted score is

$$Q_i(\tau, \Delta) = \begin{cases} S_i^0 R_i(\Delta), & T_i \leq \tau, \\ 0, & T_i > \tau. \end{cases} \tag{4}$$

The weighted score has the same reference unit as the initial score. For example, suppose a task has an initial reference score of ten units and is accepted before the deadline. An assessment rule might give equal weight to ten acceptance criteria. If eight are still met at the later assessment, the assessed fraction is 0.8 and the weighted score is eight reference

units. This is one possible rule for partial fulfilment. The study must choose a rule suited to its requirements and explain how intentional replacement is distinguished from loss caused by a defect. Lines of code that remain in place, tests that pass and an absence of reported incidents each give only partial evidence that the requirement is still satisfied. The measurement pilot must check which defects the assessment detects and how closely independent reviewers agree.

Dividing the sum of weighted scores by total human labour gives the productivity ratio, measured in reference units per person-hour. For a specified finite set of tasks with positive total labour, define

$$P_H(\tau, \Delta) = \frac{\sum_i Q_i(\tau, \Delta)}{\sum_i H_i(\tau + \Delta)}. \tag{5}$$

The denominator counts labour through the common task age $\tau + \Delta$ for every task in that set. It therefore includes labour on rejected, unfinished and abandoned tasks, together with work after acceptance that is assigned to the task. Within this common period, an early acceptance leaves more time for repair than a late acceptance. Counting repair only until Δ after each acceptance would make the end of labour counting depend on acceptance time and define a different denominator. The ratio used here includes all labour assigned to each task through task age $\tau + \Delta$.

An individual task can contribute a positive weighted score with zero human labour. The ratio requires positive labour only across the task set as a whole. Report both totals alongside the ratio so readers can see which quantity drives any difference.

3.3. **The acceptance deadline and observation periods.** Why do the requirement assessment and the labour total use different end times? Each task accepted by the deadline is assessed after the same delay from its first acceptance. Labour is counted to the same age from the start of tracking for every task, so the total includes work on tasks still unfinished or already abandoned. The end of labour counting therefore stays independent of acceptance time.

The deadline τ determines which tasks qualify for a weighted score. For a task accepted by that deadline, assess requirement fulfilment at age $T_i + \Delta$. For every task, count labour through age $\tau + \Delta$, including work on unfinished and abandoned tasks. These calculations use first acceptance times for accepted tasks and distinguish unfinished or abandoned tasks from tasks whose later progress is unknown. They also need the requirement assessment for each eligible task and the human time assigned to every task through the end of labour counting. For example, an issue comment can confirm acceptance, an assessment note can document which acceptance criteria remain satisfied, and work diaries can supply time spent on implementation or repair.

Figure 2 shows when the accepted requirement is assessed and the period included in the labour total for an example task. Adding s_i converts task age to calendar time. This conversion lets us combine the task's labour with work on other tasks that use the same people's time.

Completing every scheduled task check can establish acceptance times and later requirement fulfilment while leaving the timing of human work inside measured intervals unresolved. The available interval measurements *point-identify* realised labour when at least one allocation fits those measurements and the model constraints, and every such allocation gives the same total. This asks how much labour occurred during the measured periods. Proposition 2.1 asks a different question, whether the distribution of commit events uniquely determines the activity-model parameters. The next section derives the smallest and largest labour totals allowed by measurements over time intervals.

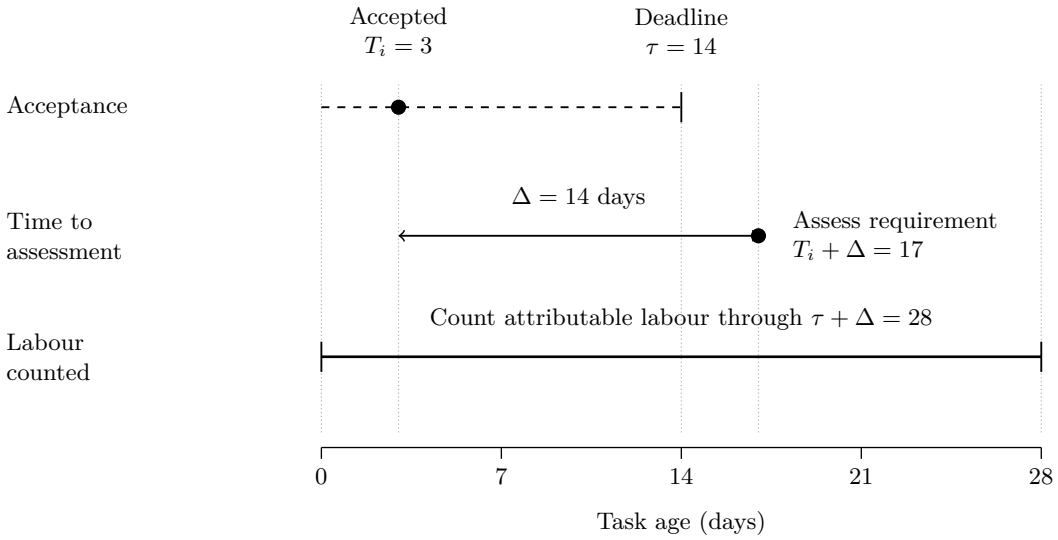


FIGURE 2. **Requirement fulfilment is assessed after acceptance, while labour is counted to a common task age.** In this example, the task is accepted on day three and both τ and Δ are fourteen days. The fraction of its accepted requirement still satisfied is assessed on day seventeen, fourteen days after acceptance. Its labour total includes work through day twenty-eight, the common task age $\tau + \Delta$, including labour attributable to the task after its requirement assessment. The solid line marks the period over which labour is summed. Human work may occupy only part of that period. Adding s_i converts task age to calendar time.

4. Attainable labour ranges from interval totals and bounds

A labour total for an interval can leave the timing of that work unknown. For each period whose labour we want to calculate, called a target window, we seek every total compatible with the supplied interval labour totals or bounds and the allocation constraint in (3).

Each *interval observation* specifies a time interval, the people, tasks and types of work included, and either their exact labour total or bounds on it. This is the meaning of "observation" in the mathematical results below. The *identified range* contains exactly the totals that allocations satisfying these constraints can produce. This definition follows the general partial-identification perspective [24, Section 1].[6] We begin with one person's work in one interval, then account jointly for overlapping intervals and shared time.

4.1. Using the person's available time.

How much of a known interval total must lie before a chosen time? Labour that exceeds the available time after that point must have occurred before it. One person can contribute at most one person-hour in an elapsed hour, so each part of the interval has a known capacity. A total across several people must be compared with their combined available time.

In the proposition, ℓ and u are the interval's start and end, c is the time dividing it, and v is the known total labour in that interval. The unknown x is the labour before c.[7]

[6] Partial identification retains all values compatible with the observations and assumptions. Point identification occurs when this set contains exactly one value. Here the values are labour totals compatible with the supplied interval totals or bounds and allocation limits. When interval labour bounds are calculated from a sample, coverage describes how often the resulting random bounds contain the true total over repeated samples.

[7] The notation $a : [\ell, u] \to [0, 1]$ assigns an allocation fraction between zero and one to each time in the interval. Square brackets include both endpoints. The functions min and max select the smaller and larger

Proposition 4.1 (Attainable contribution from one interval)**.** *Let $\ell < u$, $c \in [\ell, u]$ and $v \in [0, u - \ell]$. Among measurable allocations $a : [\ell, u] \to [0, 1]$ with $\int_\ell^u a(t)\,dt = v$, the attainable values of $x = \int_\ell^c a(t)\,dt$ are exactly*

$$[\max\{0, v - (u - c)\},\ \min\{v, c - \ell\}]\,. \tag{6}$$

If $v < 0$ or $v > u - \ell$, no such allocation exists.

Proof. Nonnegativity and capacity on the two pieces imply

$$0 \le x \le c - \ell, \qquad 0 \le v - x \le u - c.$$

Together, these inequalities give (6). Conversely, fix any x in this interval. For $\ell < c < u$, define the allocation function by

$$a(t) = \begin{cases} x/(c - \ell), & \ell \le t < c, \\ (v - x)/(u - c), & c \le t \le u. \end{cases}$$

Both allocation rates lie in $[0, 1]$ by the preceding inequalities, so the function satisfies the capacity constraint. Its integrals over the two subintervals are

$$\int_\ell^c a(t)\,dt = \frac{x}{c - \ell}(c - \ell) = x,$$
$$\int_c^u a(t)\,dt = \frac{v - x}{u - c}(u - c) = v - x.$$

Adding them gives total labour v. When $c = \ell$, the interval for x forces $x = 0$. When $c = u$, it forces $x = v$. In either case, use the constant allocation $a(t) = v/(u - \ell)$ on the complete interval, so the zero-length subinterval contributes zero without division by its length. Finally, integrating $0 \le a \le 1$ gives the necessary condition $0 \le v \le u - \ell$. □

For one person, 3/2 person-hours in a two-hour interval require between 1/2 and 1 person-hours in its first hour. The second hour can contain at most one person-hour, forcing at least half a person-hour into the first. The first hour's own capacity gives the upper bound. Capacity therefore narrows the range from $[0, 3/2]$, which nonnegativity alone permits, while leaving the exact first-hour total and timing unresolved.

If $v = u - \ell$, the total uses all the person's available time, so the labour before any time within it equals the elapsed time from its start. The same accounting extends to an interior subinterval $[b, c] \subseteq [\ell, u]$. Applying the same allocation rule inside and outside that subinterval gives its full range of labour totals, $[\max\{0, v - ((u - \ell) - (c - b))\}, \min\{v, c - b\}]$.

Why can adding separate task bounds give too wide a range? Allocations that are possible separately may be incompatible when they share a person's time. Adding lower bounds and adding upper bounds still gives valid bounds on the combined total, provided each contribution is counted once. Reaching the summed lower bound requires all component minima in one feasible allocation. Reaching the upper bound requires all component maxima together. Disjoint intervals for one person, or intervals for different people, can be combined when no other constraints link them. Labour bounds for overlapping intervals, or for tasks sharing a person's time, require a joint calculation.

For example, suppose one person spends one person-hour on each of two tasks during $[0, 2]$. Each task separately allows $[0, 1]$ person-hours before time 1. Together, the tasks use all the person's available time on $[0, 2]$, so exactly one person-hour must fall before time 1. Adding the separately calculated bounds would instead give $[0, 2]$. Figure 3 shows how the shared capacity restricts the jointly feasible first-hour totals within the separately permitted ranges. Panel (c) gives one schedule for the marked midpoint. It illustrates how consecutive activities can preserve the required totals, as proved below.

---

of their listed arguments. In this section, c is a time boundary. The same letter in Proposition 2.1 denotes a probability.

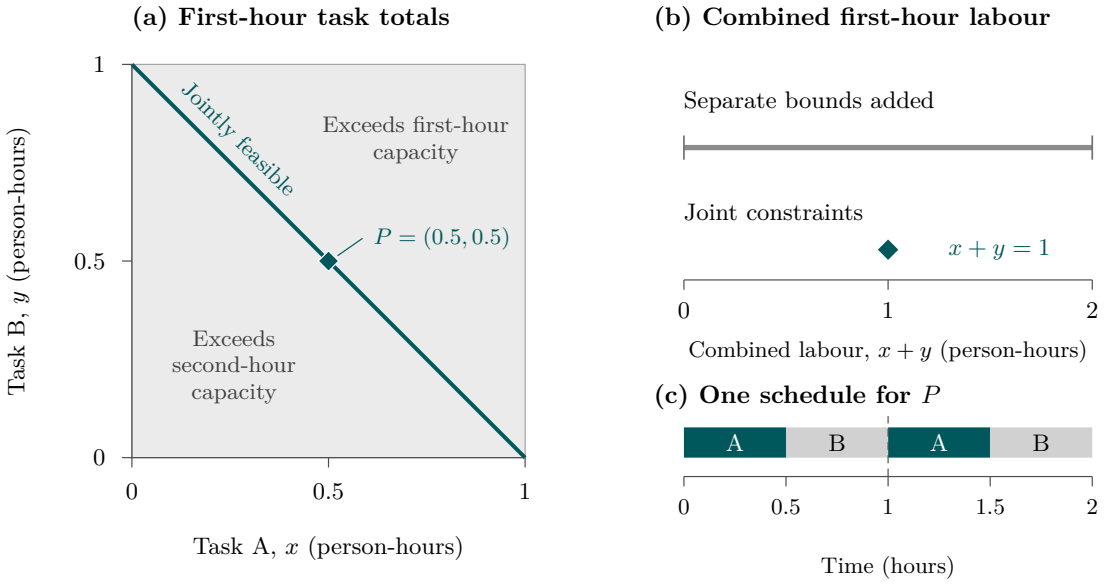


FIGURE 3. **Shared capacity determines total labour in the first hour.** In this numerical example, one person spends one person-hour on each of two tasks during two hours. In (a), x and y give the labour on each task during the first hour. Each point in the shaded square satisfies the bounds calculated for the tasks separately. Accounting for the person's available time in both hours leaves only the diagonal as possible. In (b), adding the separate bounds permits a combined first-hour total between zero and two person-hours. Accounting jointly for both tasks requires exactly one. In (c), one schedule with a single activity at a time achieves the marked point $P = (0.5, 0.5)$ by giving each task half an hour in each hour. The example allows the person to switch tasks at arbitrary times. The schedule illustrates one of the possible arrangements under these assumptions.

4.2. **Replacing allocation histories with interval totals.** The joint calculation must count all tasks drawing on the same person's available time. Call one person's work of one type on one task an allocation stream. Fix finite sets of tasks $\mathcal{I}$, people $\mathcal{J}$ and types of work $\mathcal{R}$, and select the tasks to be studied as $\mathcal{C} \subseteq \mathcal{I}$. Choose a bounded calendar-time domain $[b_0, b_K]$, with $b_0 < b_K$, containing every observation interval and target window.[8] An allocation history permitted by the model consists of measurable $a_{ijr} \geq 0$ satisfying $\sum_{i,r} a_{ijr}(t) \leq 1$ almost everywhere for every person j.

The people included and the rules for assigning time to tasks determine whose labour enters the calculation and which work it covers. Applying the model to workplace measurements requires checks that those people and attribution rules cover the intended work. Work on tasks outside $\mathcal{C}$ can have its own interval labour totals or bounds. These constrain the time available for studied tasks without adding to the target labour total.

Each interval observation constrains the labour of specified streams during one interval. For observations $\nu = 1, \ldots, M$, specify an interval $[\ell_\nu, u_\nu]$ within this domain, a set $G_\nu \subseteq \mathcal{I} \times \mathcal{J} \times \mathcal{R}$ of included streams, and finite labour bounds $v_\nu^- \leq v_\nu^+$, in person-hours. These

[8]The notation $\mathcal{C} \subseteq \mathcal{I}$ means that every studied task belongs to the full task set. The Cartesian product $\mathcal{I} \times \mathcal{J} \times \mathcal{R}$ contains every task–person–work-type triple. Vertical bars such as $|\mathcal{I}|$ count the members of a finite set. A sum indexed by a set includes exactly its members. Later, $h(\mathcal{P})$ means all target values $h(x)$ obtained from permitted vectors of cell labour totals $x \in \mathcal{P}$.

bounds impose

$$v_\nu^- \le \sum_{(i,j,r)\in G_\nu} \int_{\ell_\nu}^{u_\nu} a_{ijr}(t)\,dt \le v_\nu^+. \tag{7}$$

Exact interval totals use $v_\nu^- = v_\nu^+$. Justified limits on rounding or measurement error can give different lower and upper bounds. An estimate from sampled intervals, such as the Horvitz–Thompson estimate in Section 5, varies with the selected sample. Account for its sampling uncertainty before using it to justify either inequality.

How can we account for every possible activity history? We divide time at the domain boundaries and every endpoint of an interval carrying a labour bound or defining a target window. Keeping the labour total within each resulting interval preserves exactly the information needed for these calculations.

Sort these distinct boundaries as $b_0 < \cdots < b_K$. Call each interval between successive boundaries a cell. Every observation interval and target window then consists of complete cells, so its labour is the sum of their totals. For the productivity denominator in (5), include each studied task's tracking start s_i and labour cutoff $c_i = s_i + \tau + \Delta$ among these boundaries.

Use calendar time for this partition so work on different tasks shares the appropriate capacity constraint whenever it uses the same person at the same time. Convert task ages to calendar times first. Include cells outside a target window whenever an interval with a supplied labour total or bound spans them. Part of that interval's labour may lie outside the target window. Write $d_k = b_k - b_{k-1} > 0$ for the cell duration and x_{ijrk} for the labour in stream (i, j, r) during that cell. Each x_{ijrk} is measured in person-hours, while $a_{ijr}(t)$ is an allocation fraction. The weights in the following result select or scale cell contributions to the target. A weight of one includes a contribution and zero excludes it.

Proposition 4.2 (Exact cell reduction and attainable labour range)**.** *Under the preceding model, define the finite-dimensional feasible set $\mathcal{P}$ by*

$$x_{ijrk} \ge 0, \qquad \sum_{i,r} x_{ijrk} \le d_k \qquad \text{for every } j, k, \tag{8}$$

$$v_\nu^- \le \sum_{(i,j,r)\in G_\nu} \sum_{k:\,[b_{k-1},b_k]\subseteq[\ell_\nu,u_\nu]} x_{ijrk} \le v_\nu^+ \qquad \text{for every } \nu. \tag{9}$$

Fix real weights w_{ijrk} and define the target for a continuous-time allocation history by multiplying each cell integral by w_{ijrk} and summing. The attainable target values over histories satisfying the capacity and observation constraints equal $h(\mathcal{P})$, where $h(x) = \sum_{i,j,r,k} w_{ijrk} x_{ijrk}$. In particular, the denominator in (5) uses $w_{ijrk} = 1$ exactly when $i \in \mathcal{C}$ and $[b_{k-1}, b_k] \subseteq [s_i, c_i]$, and zero otherwise. If $\mathcal{P}$ is nonempty, its target range is the closed interval

$$[H_{\min}, H_{\max}] = \left[\min_{x\in\mathcal{P}} h(x),\ \max_{x\in\mathcal{P}} h(x)\right]. \tag{10}$$

Both endpoints and every value between them are attained, even if every $a_{ijr}(t)$ is restricted to $\{0, 1\}$, so that each person does at most one type of work on one task at a time. If $\mathcal{P}$ is empty, the observations and capacity assumptions are inconsistent.

Proof. Integrating any such history over each cell gives nonnegative cell labour totals. Integrating each person's allocation limit gives (8). Splitting each observation at the cell boundaries gives (9). The weighted target total is unchanged because its weights are constant on cells.

Conversely, fix $x \in \mathcal{P}$. For each person and cell, order the task–type-of-work pairs and divide the cell into consecutive blocks whose lengths are their respective x_{ijrk}. The capacity constraint ensures that these blocks fit inside the cell. Use half-open blocks to avoid overlap at their boundaries. During each block, set the corresponding allocation to one and all

others to zero. Leave any remaining time idle. Omit zero-length blocks and give the final domain endpoint the same allocation values as the last block. These measurable allocations reproduce every cell labour total, so all observation bounds remain satisfied and the target value is unchanged. Each person works on at most one task and type of work at a time, with finitely many changes. Thus even these restricted allocation functions give exactly the same possible target values as the finite feasible set.

The constraints define a closed convex set. Each coordinate is between zero and d_k, so a nonempty $\mathcal{P}$ is compact and a linear functional attains its extrema. Convex combinations of minimising and maximising cell vectors attain every intermediate value. Reconstructing each such vector by consecutive blocks supplies an allocation with values in $\{0, 1\}$ for each intermediate total. □

Compactness ensures that the minimum and maximum are reached. Convexity ensures that every total between them is possible.[9]

We can therefore calculate the minimum and maximum target totals using linear programmes, each with $|\mathcal{I}||\mathcal{J}||\mathcal{R}|K$ variables, $|\mathcal{J}|K$ shared-capacity constraints and $2M$ observation inequalities, in addition to nonnegativity. A known period of unavailability can be included by adding its boundaries to the partition and setting that person's cell labour totals to zero within it.

Arranging work in consecutive blocks gives each person one activity at a time while preserving the cell totals. It uses at most $|\mathcal{I}||\mathcal{R}| + 1$ constant blocks per person in each cell, including idle time. This arrangement assumes that work can start or stop at any time within a cell and that the person can switch tasks or types of work at any such time. Minimum session lengths, restrictions on interruption, task dependencies, smoothness constraints and switching costs require a separate analysis.

The schedule obtained by arranging consecutive blocks is one history of human activity consistent with the supplied interval totals and weighted labour targets. AI workflows may keep executing concurrently while the person changes task. The continuous-time history and cell totals describe the same labour quantities under the model's constraints. Establishing that such a schedule preserves the actual task order, acceptance times and requirement fulfilment would require further information and constraints. Allocations satisfying these extra restrictions form a subset of the model's feasible allocations. Their labour totals therefore lie between the calculated minimum and maximum, although a separate analysis is needed to establish which totals the restricted workflow can achieve.

Preserving the integrals while replacing fractional allocations by one activity at a time is a finite-partition instance of the Dvoretzky–Wald–Wolfowitz purification theorem [11, Theorem 4].[10] See also [18, Section 2]. The consecutive-block argument supplies the schedule directly and limits the number of blocks it uses.

4.3. **Checking the bounds and allowing error in interval labour totals.** How can a proposed minimum or maximum be checked? Give an allocation that satisfies every interval labour bound and each person's capacity constraint and achieves the proposed total. Then derive a matching bound that applies to every feasible allocation.

[9] A convex set contains every weighted average $\eta x + (1 - \eta)y$ of two of its points x and y, with weight $0 \leq \eta \leq 1$. Closedness means that limits of convergent sequences in the set also belong to the set. Boundedness places finite limits on its coordinates. In finite dimensions these two properties imply compactness by the Heine–Borel theorem. The extreme-value theorem then guarantees that the continuous weighted sum h attains a minimum and maximum on a nonempty feasible set. Convexity supplies every intermediate target value, and the proof arranges consecutive activities to realise each as a schedule.

[10] For finitely many activities and finite atomless measures, purification replaces fractional allocations by a single activity at each time while preserving the specified integrals. Here elapsed time is atomless. Every measurable set of positive duration can be divided into parts of smaller positive duration. Including idle time makes the allocation fractions sum to one. The consecutive-block proof supplies this replacement directly for the paper's finite time partition.

For a maximum, the bound must put every feasible target total at or below the proposed value. For a minimum, it must put every feasible total at or above that value. The allocation establishes that the value is attainable, and the matching bound proves that it is the optimum.

Collect the cell totals and target weights into vectors x and w, and write the capacity inequalities and interval labour bounds as $Ax \le b$, with $x \ge 0$ kept separate.[11] A lower bound on an interval labour total is included by reversing its sign. Nonnegative multipliers λ and μ satisfying

$$A^{\mathsf{T}}\lambda \ge w, \qquad A^{\mathsf{T}}\mu \ge -w \quad \Longrightarrow \quad -\mu^{\mathsf{T}}b \le h(x) \le \lambda^{\mathsf{T}}b \qquad (x \in \mathcal{P}) \tag{11}$$

give bounds by taking weighted sums of the original inequalities, yielding $w^{\mathsf{T}}x \le \lambda^{\mathsf{T}}Ax \le \lambda^{\mathsf{T}}b$, and likewise for $-w$.

For a nonempty $\mathcal{P}$, linear-programming strong duality ensures that suitable multipliers give exactly $H_{\min}$ and $H_{\max}$ [6, Sections 5.2–5.3].[12] When the bound calculated from the multipliers equals the total achieved by a feasible allocation, together they prove that the total is the minimum or maximum. If the problem data, cell totals and multipliers are rational, these checks can use exact arithmetic without repeating the optimisation. The allocation and multipliers therefore let us check the minimum or maximum reported by a numerical solver.

With the partition and target fixed, widening the bounds on interval labour totals changes only the right-hand sides of the inequalities. Increasing an upper labour bound or decreasing a lower one increases the corresponding entry of b. If these changes give $b + \varepsilon$, with $\varepsilon \ge 0$ and zero entries for unchanged constraints, the original multipliers still bound every newly permitted target total by

$$[-\mu^{\mathsf{T}}(b+\varepsilon),\ \lambda^{\mathsf{T}}(b+\varepsilon)]. \tag{12}$$

The same weighted sums of inequalities give these bounds, as in the sensitivity analysis of [6, Section 5.6]. Solving the optimisation again may give a narrower interval. Evidence about the labour measurements and any rounding must still justify the size of those error limits.

Bounds estimated from a sample require an argument about how often they contain the true totals. Suppose random lower and upper bounds contain all the true interval labour totals simultaneously with probability at least $1 - \alpha$, where $0 < \alpha < 1$. If the true history satisfies the allocation constraints, its cell vector belongs to the resulting random feasible set on that event. Optimising then gives an interval containing the true target total with probability at least $1 - \alpha$. An empty feasible set counts as a failure to cover.

This coverage claim concerns all interval labour totals together. It therefore needs a justification based on the actual sampling and measurement process. Coverage of $1 - \alpha$ for each interval total separately does not establish the same probability of covering them all. The union bound supplies one such justification when the individual failure probabilities sum to at most α.[13] The resulting interval has a sampling-coverage guarantee, rather than the exact-data interpretation of (10).

[11] Each row of A contains one constraint's coefficients, and the matching entry of b is its limit. Vector inequalities compare corresponding entries. The superscript T denotes transpose, so $w^{\mathsf{T}}x$ sums the products of matching weights and cell totals. The vectors λ and μ give nonnegative weights to constraints. The arrow $\Longrightarrow$ denotes implication.

[12] Weak duality gives a valid bound from each feasible choice of multipliers. Strong duality states that the best such bound equals the optimum of the original linear programme. Here feasibility and bounded cell totals give finite attainable optima, and linear-programming duality supplies matching multipliers. Rational numbers are ratios of integers. When the data and proposed solution are rational, their equalities and inequalities can be checked using exact fraction arithmetic.

[13] For events $E_1, \ldots, E_M$, the union bound is $\mathbb{P}(\bigcup_{\nu=1}^{M} E_\nu) \le \sum_{\nu=1}^{M} \mathbb{P}(E_\nu)$. The union denotes the event that at least one E_ν occurs. Taking E_ν to be failure of the labour bounds for interval ν limits the probability of any failure by the sum of the individual failure probabilities. This inequality holds regardless of dependence between the events.

A separate Lean 4 development using mathlib [9, 23] checks the steps from interval integrals to cell totals and from each person's allocation limit to cell capacity. It also checks schedules with at most one human activity at a time while preserving the supplied interval totals and weighted labour targets. Concurrent AI execution remains in the separate computing-resource account described in Section 3.

The Lean development further checks bounds from supplied allocations and dual multipliers, and the implication from simultaneous coverage of interval labour totals to coverage of the target total. Its scope excludes the existence of minimising and maximising allocations, the existence of matching dual multipliers and the limit on the number of schedule blocks stated above. The Rust solver is tested separately. Correspondence between its source code and the Lean definitions remains outside the formal proof.

4.4. **From labour bounds to productivity.** Use the labour range corresponding to the denominator in (5). Suppose the sum of weighted scores at the follow-up assessments, $Q = \sum_{i \in \mathcal{C}} Q_i(\tau, \Delta)$, is known exactly. With $Q \geq 0$ and $H_{\min} > 0$, dividing this score by the largest and smallest possible labour totals gives the identified productivity interval $[Q/H_{\max}, Q/H_{\min}]$. The labour bounds appear in reverse order because dividing a fixed nonnegative score by more labour gives the same or a smaller productivity ratio.

When the total score is also uncertain, separate score and labour bounds leave open which pairs can occur together.[14] Justified bounds on the total score $0 \leq Q_- \leq Q \leq Q_+$ give an interval $[Q_-/H_{\max}, Q_+/H_{\min}]$ containing every feasible ratio of total score to person-hours. It is the exact range only when every ratio it contains comes from a total score and labour total allowed together by all the constraints on these quantities. It is sufficient for every pair in $[Q_-, Q_+] \times [H_{\min}, H_{\max}]$ to be permitted. A lower labour bound of zero cannot be used as a divisor. The labour optimisation varies allocations while holding acceptance times and the assessed fractions of requirements still satisfied in (4) unchanged.

5. Estimating labour from sampled allocations

The preceding bounds describe every labour total permitted by a given set of interval observations. What changes when labour is measured only in randomly selected intervals for individual people? We estimate total labour and assess how the estimate varies across the possible samples. Treating that estimate as an exact interval total can exclude the true allocation through sampling error, so its uncertainty needs separate analysis. The difference estimator below corrects a predicted total using measured labour from the selected intervals. Under its sampling and measurement assumptions, the corrected estimate averages to the true total over repeated samples.

The sampling frame is the complete list of time intervals for individual people from which the sample is selected. It must cover all work included in the labour total, including reasoning away from the keyboard as well as review and coordination. For a selected interval, a short work diary can identify the person and time period, then give the minutes spent on each task and type of work. These durations supply the labour measurements used by the estimator. Check a subset using a separately obtained account of the same intervals. For example, an observer could time activities and compare those durations with the participant's diary. Explain who made each measurement and how differences were investigated, and report any disagreement that remains. Missing diary entries leave selected intervals unmeasured, while inaccurate recall can distort the durations supplied. Work omitted from the sampling frame requires a separate check of whether its list covers the intended labour total.

[14] These bounds describe which pairs of total score and labour the supplied constraints permit, including any restriction linking the two quantities. A probability model would also give probabilities for ranges of paired values. In such a model, a copula combines the separate distributions of total score and labour to describe their statistical dependence, as formalised by Sklar's theorem [13, Theorem 2]. The present calculation uses feasible pairs directly. Allowing every pair in the two ranges is a condition on possible values, whereas statistical independence is a condition on their probabilities.

5.1. **Predictions calculated before sampling.** How can predictions with errors still give an unbiased estimate of total labour? We estimate their combined error from selected intervals and add that correction to the predicted total. Under the conditions below, the corrected estimate averages to true labour over repeated samples.

Sum the predictions for all person-intervals to obtain a predicted labour total. In each selected person-interval, subtract predicted person-hours from measured person-hours to obtain the residual. Divide that residual by the interval's selection probability, then add the weighted residuals to estimate the correction for the complete set of intervals. Denis and Haddou give this estimator, with predictions held unchanged across possible samples, and its sampling variance [10, Section 2.3, equations (2)–(4)]. Prediction-powered inference likewise identifies its estimator of a mean as a difference estimator and considers finite-population extensions [3, Section 1.4 and Appendix B of the accompanying preprint].

For each person, divide the period whose labour we want to estimate into non-overlapping intervals. Let $U = \{1, \dots, M\}$, $M \geq 1$, be the complete list, or sampling frame, of these person-intervals. This list stays the same whichever sample is selected. For interval m, let $d_m > 0$ be its length in hours and $a_m \in [0, 1]$ the average fraction of the person's time spent on the included work. Its labour is $y_m = d_m a_m$ person-hours, and total labour over the complete list is $H = \sum_U y_m$.[15]

Calculate a prediction f_m in person-hours for every interval in the frame. Convert Standard Coding Hours to predicted person-hours before using them here, and state the conversion calculation and the evidence supporting it. Measured and predicted labour must cover the same work and time period. Each prediction must remain unchanged whichever intervals are selected to measure labour and correct the predicted total. Training on independent earlier data can meet this condition when all predictions are calculated before selection and then kept unchanged.

Proposition 5.1 (Fixed-prediction difference estimator)**.** *For a probability sample $s \subseteq U$ with known first-order inclusion probabilities $\pi_m > 0$, set $r_m = y_m - f_m$ and*

$$\widehat{H}_{\text{diff}} = \sum_{m \in U} f_m + \sum_{m \in s} \frac{y_m - f_m}{\pi_m}. \tag{13}$$

If y_m and f_m are fixed under the design and every sampled labour value is observed accurately, then $\mathbb{E}_s[\widehat{H}_{\text{diff}}] = H$, where $\mathbb{E}_s$ averages over the sampling design.

Proof. Write $I_m = \mathbf{1}\{m \in s\}$. Linearity and $\mathbb{E}_s[I_m] = \pi_m$ give

$$\mathbb{E}_s[\widehat{H}_{\text{diff}}] = \sum_U f_m + \sum_U \frac{\mathbb{E}_s[I_m]}{\pi_m}(y_m - f_m) = \sum_U y_m.$$

□

In the proof, I_m marks whether interval m was selected. Dividing the residual calculated for that interval by π_m makes its average contribution over possible samples equal the interval's full residual. The corrected estimate therefore averages to total labour even when the predicted total differs from H. Taking $f_m = 0$ gives the Horvitz–Thompson estimator, labelled HT in the equations [14],

$$\widehat{H}_{\text{HT}} = \sum_{m \in s} \frac{d_m a_m}{\pi_m}. \tag{14}$$

[15]The sampling design specifies the probabilities of selecting the possible samples from this fixed frame. The inclusion probability π_m is the probability that interval m is selected. The indicator $\mathbf{1}\{m \in s\}$ is one when it is selected and zero otherwise. A hat, as in $\widehat{H}$, marks an estimate. The subscript s on $\mathbb{E}_s$ or Var_s means that the average or variance is taken over possible samples, with labour values and predictions held fixed.

For every person-interval in the sampling frame, record whether it was selected. For a selected interval, keep the measured fraction of that person's time spent on the included work, including zero, or mark that measurement as missing. A zero allocation measurement assigns none of that interval to the included work. A missing response leaves the allocation unknown in the supplied data. The proof uses true allocations, so a claim of unbiasedness in the presence of measurement error requires additional assumptions about that error.

One possible study format is a table with a row for each person-interval. The row could identify the person and the interval's start and end times, give its selection probability and state whether it was selected. For a selected interval, the row would also contain either the measured allocation or a missing-response status. This proposed format makes the distinction between a measured zero and a missing measurement explicit.

5.2. **Precision depends on residuals and the sampling design.** Why can an unbiased estimate still be imprecise? Unbiasedness concerns the average across possible samples. Precision concerns how closely the individual estimates cluster around that average. Their spread depends on both the residuals and the way intervals are selected.[16] Let $\pi_{mk} = \mathbb{E}_s[I_m I_k]$ be the probability that intervals m and k are both selected, including $\pi_{mm} = \pi_m$. The exact variance over repeated samples is

$$\operatorname{Var}_s(\widehat{H}_{\text{diff}}) = \sum_{m,k \in U} \frac{\pi_{mk} - \pi_m \pi_k}{\pi_m \pi_k} r_m r_k. \tag{15}$$

Expanding the variance of $\sum_U I_m r_m / \pi_m$ gives this expression. The pair terms account for the dependence between selections. Equation (15) uses residuals for the whole frame, including unsampled intervals whose residuals are generally unknown. Estimating the variance therefore requires the actual sampling design and the joint selection probabilities required by the chosen variance estimator.

For simple random sampling without replacement from $M \geq 2$ units, let the fixed sample size satisfy $1 \leq n \leq M$. Every size-n subset has the same probability and $\pi_m = n/M$. Let $\bar{r}_U$ and $\bar{y}_U$ be the means of the residuals and labour values over the complete frame. Their finite-population variances are

$$S_r^2 = \frac{1}{M-1} \sum_U (r_m - \bar{r}_U)^2, \qquad S_y^2 = \frac{1}{M-1} \sum_U (y_m - \bar{y}_U)^2.$$

Substituting the selection probabilities for this sampling design into (15) gives

$$\operatorname{Var}_s(\widehat{H}_{\text{diff}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_r^2, \qquad \operatorname{Var}_s(\widehat{H}_{\text{HT}}) = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) S_y^2. \tag{16}$$

The factor $1 - n/M$ accounts for sampling without replacement and becomes zero when every interval is measured. To estimate the variance from the sample, let $\bar{r}_s$ be its mean residual and s_r^2 its residual variance. For $2 \leq n \leq M$, an unbiased estimator of the sampling variance is

$$\widehat{V}_{\text{diff}} = \frac{M^2}{n} \left(1 - \frac{n}{M}\right) s_r^2, \qquad s_r^2 = \frac{1}{n-1} \sum_{m \in s} (r_m - \bar{r}_s)^2. \tag{17}$$

The identity $s_r^2 = [n(n-1)]^{-1} \sum_{m<k,\, m,k \in s} (r_m - r_k)^2$ expresses sample variance through pairwise differences. Each pair has inclusion probability $n(n-1)/[M(M-1)]$, so $\mathbb{E}_s[s_r^2] = S_r^2$. A sample containing one interval from a larger frame cannot supply this variance estimate. Measuring every interval gives zero sampling variance, including when the frame contains just one interval. Inaccurate allocation measurements can still change the labour estimate, so measuring every interval also requires checks of measurement accuracy.

[16] Variance is the expected squared distance from the mean. Here it has units of squared person-hours, and its square root has units of person-hours. Covariance describes how two random quantities vary together. For selection indicators, $\pi_{mk} - \pi_m \pi_k$ is their covariance, where π_{mk} is the probability of selecting both intervals.

For $n < M$, both variances have the same positive sampling factor. Predictions therefore improve precision exactly when $S_r^2 < S_y^2$, meaning that the residuals vary less across intervals than the labour values themselves. An unbiased correction alone gives no assurance of this reduction.

For example, take $M = 4$, $n = 2$, and $y = (0, 2, 4, 6)$. True total labour is 12, and the Horvitz–Thompson variance is $80/3$. Holding $f = (1, 3, 3, 5)$ fixed across samples reduces the variance to $16/3$. Holding $f = (6, 4, 2, 0)$ fixed instead raises it to $320/3$. This second prediction ranks the intervals in reverse order of their labour, increasing residual variation and giving four times the variance of estimation without predictions. Both corrected estimators remain unbiased.

Figure 4 shows the estimate from each of the six equally likely samples. All three sets of estimates average to the same true total, while their spreads differ.

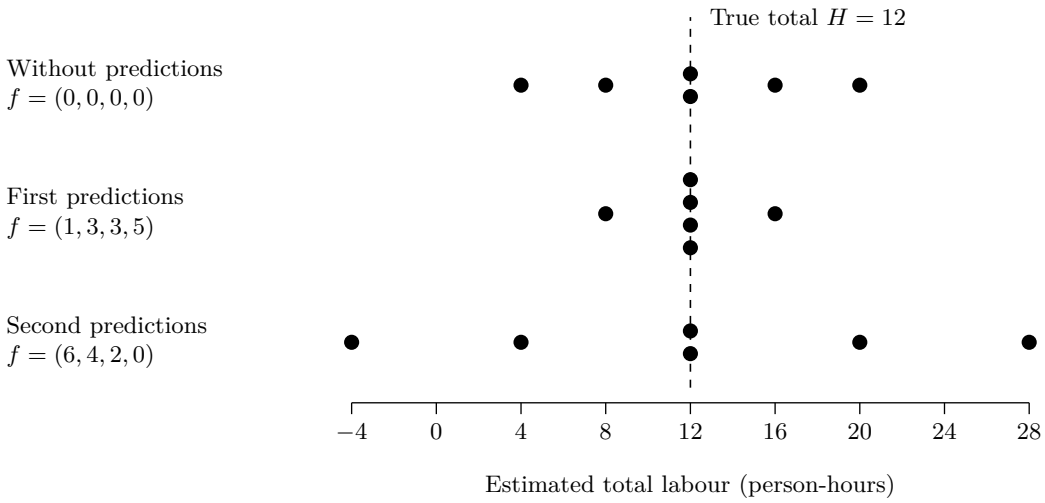


FIGURE 4. **The estimates have the same average but vary differently across samples.** For the hypothetical labour values $y = (0, 2, 4, 6)$, each dot shows the estimated total from one of the six equally likely samples of two person-intervals, meaning time intervals for individual people. Dots at the same value are stacked so every sample is visible. Each row averages to the true total of twelve person-hours. Predictions $f = (1, 3, 3, 5)$ reduce variation across samples relative to estimation without predictions. Predictions $f = (6, 4, 2, 0)$ increase that variation and can produce a negative estimate. Each row shows all six samples and the estimate calculated from each.

Calculating all six samples reproduces these variances and confirms that each estimator averages to the true total. It also confirms that the variance estimate in (17) averages to the corresponding true variance. These checks establish the calculations for the two specified predictors. A workplace study must separately test whether its code-change predictions leave a total residual that can be estimated more precisely than labour itself.

5.3. **Failure cases and implementation checks.** Why must predictions stay the same whichever intervals are selected for correction? The proof averages over possible samples while holding every prediction unchanged. Training on the selected intervals lets the predictions change with the sample and can change the estimate's average. For $M = 2$, $n = 1$ and $y = (0, 2)$, suppose a fitted rule memorises the selected labour value and predicts zero for the other interval. The residual calculated in the selected interval is always zero, so the correction leaves the fitted total unchanged. The resulting difference estimate is 0 or 2, each with probability $1/2$, giving expectation 1 rather than $H = 2$. A predictor trained on the

sampled intervals therefore needs its own unbiasedness argument for the actual sampling design.

Replacing negative estimates with zero can also change the estimate's average over possible samples. The estimate can be negative even though true labour is nonnegative. With $y = (0, 0)$, $f = (10, 0)$ and one uniformly selected unit, (13) is -10 or 10 with equal probability. Replacing the negative estimate with zero changes the expectation from 0 to 5. Reporting the calculated value, including its sign, preserves the estimator in (13) and exposes an implausible result for investigation.

A productivity ratio needs its own bias analysis. Dividing the sum of weighted scores by an unbiased labour estimate can still give a biased ratio because division is nonlinear.

The implementation keeps the estimate's calculated sign and preserves allocation measurements equal to zero. It distinguishes missing allocation measurements for selected person-intervals from person-intervals that were not selected. Inputs are rejected when required information is missing, when selected intervals lack allocation measurements or when supplied values are inconsistent. For a general sampling design, it reports a total-labour estimate. For simple random sampling, it also reports variance and standard error when the conditions above hold.[17]

These checks establish whether the supplied sampling frame and allocation measurements satisfy the implementation's input rules. Using the unbiasedness result with workplace data also requires evidence that the frame covers the intended work, that the allocations are measured accurately and that the supplied selection probabilities match the procedure actually used to select intervals.

Exhaustive finite tests cover the examples and input validation, including samples containing every interval and populations consisting of a single interval. A confidence interval requires a separate justification of how often it contains true labour under the sampling design.

6. Validating the measurements

Applying these calculations to workplace labour requires measurements that cover the included activities and accurately describe each person's time on them. A measurement pilot must test this coverage and accuracy before the measures are used to compare AI tools. Define the included work and the rules for assigning hours to tasks, then list the time intervals for individual people available for sampling. Before collecting labour measurements or assessing tasks, specify the reference scoring rule, acceptance deadline and criteria for judging requirement fulfilment. Together these choices determine which human work is counted and how delivered requirements are scored.

The pilot must show how often selected intervals receive usable labour measurements and how much time participants and researchers spend supplying and checking them. Include periods with concurrent AI workflows when checking how shared oversight and attended waiting are assigned under the inclusion rules. Report missing diary entries and disagreements about time allocations. Check how many eligible tasks receive their requirement assessment at the scheduled time, and measure the time needed to complete this follow-up. Summarise these measurement checks by comparison group, type of work and defined task category. They determine the collection effort and precision that a later comparison can support.

The validation checks must match the result we intend to use. Labour bounds need independently checked interval totals and justified limits on measurement error. Choose the periods for which labour will be bounded before inspecting the study's labour totals, acceptance times or weighted scores. For each period, compare the sum of separately

[17]The estimated standard error is the square root of the estimated sampling variance. It expresses the sampling spread in person-hours. A confidence interval also needs a justification for how often the procedure covers the true total. A standard error alone supplies no particular coverage probability.

calculated bounds with the minimum and maximum from the joint calculation. This shows how much accounting for each person's shared time narrows the labour range.

For sampled labour, assess whether predictions improve precision by comparing variance with and without them under the same sampling design. Calculate predictions before sampling and keep them unchanged whichever sample is drawn. Retain the selection probabilities and the information about joint selection needed to calculate uncertainty. The next subsection explains how to account for this sampling variation when assessing prediction accuracy.

6.1. **Comparing predictions with sampled labour.** What does a difference between predicted labour and a sample-based labour estimate tell us? It reflects prediction error together with variation caused by which intervals were selected. To evaluate predictions on work outside the training data, use teams or periods excluded from training. Calculate predictions before selecting the validation intervals and keep them unchanged whichever sample is selected. Under these conditions, the following calculations distinguish error against true labour from variation due to sampling.

Treat each task's labour H in a finite set of validation tasks as fixed. Let f be its prediction, calculated before validation sampling and unchanged whichever sample is drawn, and suppose $\mathbb{E}_s[\widehat{H}] = H$ with finite sampling variance. Then

$$\mathbb{E}_s[f - \widehat{H}] = f - H,$$
$$\mathbb{E}_s[(f - \widehat{H})^2] = (f - H)^2 + \mathrm{Var}_s(\widehat{H}).$$

Expanding the squared difference proves the second identity because the cross term has expectation zero when $\mathbb{E}_s[\widehat{H} - H] = 0$. Averaging the signed difference over repeated samples therefore gives the prediction's error against true labour. Averaging its square gives squared prediction error plus sampling variance. These identities hold for each task under the procedure used to select its intervals for labour measurement. Uncertainty in results combined across tasks also depends on how their labour estimates vary together.

For a second prediction g calculated before the same validation sample and unchanged whichever sample is drawn, subtracting the expected squared errors cancels the common sampling variance, giving

$$\mathbb{E}_s[(f - \widehat{H})^2 - (g - \widehat{H})^2] = (f - H)^2 - (g - H)^2.$$

Over repeated validation samples, the average difference between the two squared prediction errors equals their difference calculated against true labour. Estimating either predictor's squared error against true labour, or quantifying sampling uncertainty in their comparison, requires further sampling-design information.

Absolute prediction error and coverage of prediction intervals need a separate check because the squared-error calculation does not justify them. To examine these measures, consider eight one-hour person-intervals with allocations $(1, 1, 1, 1, 0, 0, 0, 0)$. Sample one uniformly, with inclusion probability $1/8$. True labour is $H = 4$, whereas its Horvitz–Thompson estimate is 0 or 8, each with probability $1/2$. A prediction equal to true labour, $f = 4$, therefore differs from either sampled estimate by 4. The prediction interval $[3.5, 4.5]$ always contains true labour but never contains the sampled estimate. Here the discrepancy comes entirely from sampling error in the labour estimate. The signed difference between prediction and sampled estimate remains unbiased for the prediction's error against true labour. Absolute errors and prediction-interval coverage judged against $\widehat{H}$ measure agreement with the sampled estimate, including its sampling error.

Prediction validation must distinguish prediction error from uncertainty in the labour measurements. One option is to select tasks for validation and measure labour in every person-interval within the periods being evaluated. Check those measurements independently, stating who reviewed them, which evidence resolved discrepancies and how the review was independent of the people who supplied the original allocation measurements. Report any

remaining measurement error. Alternatively, use an explicit model linking the available measurements to true task labour, under which those measurements uniquely determine the chosen accuracy measure. Keep the probabilities of selecting intervals singly and together so uncertainty can be calculated. Report systematic overstatement or understatement and individual prediction errors by study group, type of work and defined task category. Report these individual errors alongside any correlation between group means of predicted labour and the labour estimates obtained in validation.

Validate the weighted scores by checking each task's reference score and whether first acceptance occurred by the deadline. For tasks accepted by that deadline, also check the fulfilment fraction assessed at follow-up. Check whether splitting a task, squashing code changes or changing the scoring inputs changes its reference score. Report how often reviewers disagree and how many scheduled follow-up assessments were completed, alongside prediction error. Recalculate group comparisons for plausible amounts of missing labour and plausible values of disputed weighted scores. If those measurement errors change which group performs better, choosing between tools and working practices requires resolving or explicitly accepting that uncertainty.

6.2. **Conditions for comparing AI tools and working practices.** Accurate labour measurements and requirement assessments allow groups to be compared. When can a difference be attributed to AI access? The groups must support a comparison that separates the effect of AI access from other differences in their work and working conditions. AI adoption can change which work is requested or undertaken, so the study must first define which work enters the comparison. A study of tasks specified before group assignment answers a different question from a study of incoming organisational work.

For tasks specified in advance, decide which qualify before assigning tasks, people or teams to groups, and keep those inclusion rules unchanged. State how earlier work is counted. In the request list, show the status of postponed, rejected and abandoned requests and identify which belong to the included task set. An organisation-wide study instead needs a common calendar period, a list of requests received during that period and the tasks already in progress at its start. Each design must identify the population it concerns and explain how measurements are weighted to represent it.

The comparison groups must reflect how teams share work. Reviewers and reusable components can carry the effects of one team's AI access into another team's work. Shared computing resources can also link outcomes when teams compete for capacity. Account for these links when forming groups and analysing outcomes. Analyse teams according to their original group assignment and log actual tool use separately.

Calculate uncertainty in the estimated effect of AI access using the units assigned to groups, such as teams or individual people. More commits within a team increase the number of commit events but leave the number of independently assigned teams unchanged. The uncertainty calculation must also include the effects of measuring labour and checking requirement fulfilment in subsets.

Apply the same criteria for requirement fulfilment and the same follow-up assessment procedure to both groups. An assessment note can identify the task and requirement version checked, the assessment time and the reviewer's judgement. Have two reviewers assess a subset independently so their disagreement can be measured. Reviews can prompt repair, so include the time feedback reaches the team and whether the reviewer knew the task's AI-group assignment in the assessment note. Count any resulting repair within the period used to calculate labour for the productivity ratio. Estimating the sum of weighted scores from assessments of a subset of tasks requires known selection probabilities and assumptions about the review's effect on later work. Report severe defects separately because the fraction of a requirement still satisfied gives only part of the evidence about security and reliability.

Interpret any apparent labour saving alongside acceptance and weighted scores. Follow every task through $\tau + \Delta$ as required by Section 3, distinguishing permanent abandonment

from losing contact with a team or ceasing to track a task. When labour measurements, acceptance times or later requirement assessments are missing, state the assumptions or bounds used for those unknown quantities. Reduced labour may reflect more abandonment, so report task acceptance and both score and labour totals. The productivity ratio also needs uncertainty in its numerator and denominator, including how they vary together.

Before inspecting the study's labour totals, acceptance times or weighted scores, decide what improvement would be useful and what loss of requirement fulfilment would be acceptable. Set the improvement thresholds and the maximum acceptable quality reduction, called the noninferiority margin. To show that a loss falls within this margin, estimate its size and uncertainty. An undetected reduction may still exceed the margin. The following fixed-workload experiment makes both decision criteria explicit.

6.3. **A proposed causal experiment.** A fixed-workload experiment would compare teams given a specified set of AI tools and working practices with teams using the same ordinary development and review process without generative AI. Before assigning teams to groups, fix each team's task list, requirement versions, acceptance criteria and reference scores. The comparison then covers work specified before AI access is assigned. Pair teams by repository domain, previous workload and experience. Within each pair, use an independent fair coin to choose which team receives AI access [16]. Teams remain in their assigned groups throughout the study. Compare them according to this assignment regardless of actual tool use. Use task diaries or tool-usage logs to describe which tools they actually used.

The team boundaries also need to reflect how work is shared. Shared reviewers, developers, computing capacity or code can make one team's labour or requirement fulfilment depend on another team's AI access [15]. If these links require allocating several teams together, revise the estimator weights and planning before randomisation so that each team still receives equal weight in the comparison.

Each team's initial reference scores must have a positive total. Keep that total unchanged and use it as the denominator for both labour through task age $\tau + \Delta$ and the sum of weighted scores. Include labour on failed and abandoned tasks. The resulting team measures are labour per reference unit and weighted score per reference unit. The second ratio is a reference-score-weighted average of the assessed fulfilment fractions, with deadline misses contributing zero. The primary effects are the mean differences between groups in these measures, with equal weight for each team. Estimate each effect by subtracting the comparator team's measure from the AI team's within each pair, then averaging the pair differences.

A benefit claim requires both a worthwhile reduction in labour per reference unit and an acceptable loss in the score ratio. Set these thresholds before inspecting the team measures. Evidence that fails to detect a decrease in the score ratio can still allow a loss exceeding the maximum [22].

The provisional criteria require a saving exceeding one person-hour per reference unit and a loss of less than five percentage points in weighted score per reference unit. For uncertainty and measurement bias, the protocol uses approximate one-sided 97.5% Student-t bounds from independent pair differences. The bias allowances expand the labour bound by 0.2 person-hours per reference unit and the score-ratio bound by 0.01. Both expanded bounds must meet their respective criteria.[18] The measurement pilot must test whether time measurements and requirement assessments justify these bias allowances. The practical

[18]The Student t distribution accounts for estimating the standard deviation from the sample. With p pairs, an upper bound for the mean difference is the sample mean plus $t_{p-1,0.975}$ times its estimated standard error. Here $p-1$ is the degrees of freedom and $t_{p-1,0.975}$ is the distribution's 97.5th percentile. Exact Student t coverage assumes independent, identically distributed normal pair differences. The protocol uses this reference distribution as an approximation. The added allowances widen the bounds for measurement bias and need separate empirical justification.

thresholds must be agreed before the main study. This plan permits a benefit claim only when both measures are available for every team.

The accompanying protocol specifies a separate six-pair measurement pilot. The acceptance deadline and the delay from acceptance to assessment are provisionally set to $\tau = \Delta = 336$ hours (14 calendar days each). Twenty thousand simulated experiments per design and scenario compare 48 and 96 teams using hypothetical distributions of the two team measures, measurement bias and missing measurements, including cases at each benefit boundary [19]. In the required hypothetical scenarios with a worthwhile AI benefit, neither team count achieves the required 90% probability of meeting both criteria.

Before choosing the main study's sample size, the pilot must establish defensible ranges for measurement error and variation between teams. It must also establish how often both team measures can be obtained. The pilot's estimated mean effect of AI access must not determine that choice [1]. The experiment is specified but has not been run.

7. Results and next measurement checks

Different distributions of human activity can produce identical commit distributions. Interval totals, or justified bounds on them, restrict the labour possible in a specified period when combined with each person's available time. Dividing time at the start and end of every observed interval and every period whose labour total is sought turns these restrictions into a finite optimisation problem with exactly the same possible labour totals. Accounting jointly for tasks that share a person's time can exclude totals allowed by adding separate bounds. The summed lower bound is attainable only if all component minima can occur together. The same requirement applies to the upper bound and component maxima.

Selecting intervals at random gives an unbiased labour estimate when the sampling frame covers the work, selection probabilities are known and positive, and sampled allocations are measured accurately. Predictions calculated before selection and held unchanged across possible samples improve precision only when estimating their total error has lower sampling variance than estimating labour without predictions. Applying either result requires evidence that the measured durations cover the intended work and assign each person's time accurately. For example, a work diary must include reasoning and coordination as well as implementation. The measurement pilot must test that coverage and compare reported durations with separately obtained time measurements.

Establishing an AI benefit requires a causal comparison of tools and working practices. An organisation-wide adoption study would include changes to incoming requirements, whereas the proposed experiment concerns a fixed workload. Its next empirical step is the measurement pilot in Section 6.3. The pilot must test whether work diaries or other time measurements cover the included activities, and whether reviewers can assess accepted requirements at the scheduled follow-up times. Disagreement about measured time or requirement fulfilment, missing measurements and collection effort will guide the measurement procedure and the size of the main experiment.

Appendix A. A shortest-path solution for one allocation function

For one person's work of one type on one task, let $0 \le a \le 1$ be the allocation function. The interval labour bounds in Section 4 constrain its integral over each interval for which a total or bound is supplied. Subtracting cumulative totals gives the labour between two times. We can represent a bound on that difference by an edge in a graph. Using the sorted time boundaries and cell lengths defined there, set $z_k = \int_{b_0}^{b_k} a(t)\,dt$ for the cumulative labour up to boundary b_k. Represent an inequality $z_q - z_p \le w$ by a directed edge $p \to q$ of weight w.

The graph combines capacity limits with the supplied interval labour bounds. Adjacent time boundaries receive edges $k-1 \to k$ of weight d_k and $k \to k-1$ of weight zero, encoding the maximum available time and nonnegativity. Labour bounds on $[b_p, b_q]$ add edges $p \to q$

of weight v^+ and $q \to p$ of weight $-v^-$. An exact supplied labour total v therefore gives edge weights v and $-v$. This is the standard difference-constraints representation [17, Slides 6–11].

Why does a shortest path give a labour bound? Adding the inequalities along a path gives an upper limit on the difference between its final and initial cumulative totals. The path with the smallest sum of edge weights gives the tightest upper limit. A reverse path bounds the opposite difference and therefore gives a lower limit on the target labour.[19]

Proposition A.1 (Attainable cumulative bounds from graph distances)**.** *The observations for this single allocation function admit an allocation if and only if this graph has no directed cycle of negative total weight. When such an allocation exists, let $D(p, q)$ be the shortest-path weight from p to q. For any endpoint pair $p \le q$, the identified range of $\int_{b_p}^{b_q} a(t)\,dt$ is*

$$[-D(q, p),\ D(p, q)]. \tag{18}$$

Proof. Summing edge inequalities around a cycle gives 0 at the left, so a negative cycle is inconsistent. Conversely, adjacent edges make every vertex reachable from every other. In the absence of negative cycles, all shortest-path weights are finite and $D(r, r) = 0$. For any starting vertex r, the values $z_k = D(r, k)$ satisfy each edge inequality because $D(r, v) \le D(r, u) + w$ for an edge $u \to v$ of weight w. Subtracting z_0 makes initial cumulative labour zero without changing differences. The inequalities for adjacent endpoints give cell labour totals in $[0, d_k]$. Allocations constant within each cell can produce these totals and satisfy the observations.

Summing inequalities along paths yields $z_q - z_p \le D(p, q)$ and $z_q - z_p \ge -D(q, p)$. Using p as the starting vertex gives an allocation achieving the upper bound. Using q gives an allocation achieving the lower bound. Convex combinations of these allocations give every labour total between those bounds. □

The reference checker implements these bounds with exact rational arithmetic and an all-pairs shortest-path calculation. Given indexed, sorted time boundaries, graph construction and this calculation use $O((K + 1)^3 + M)$ arithmetic operations and $O((K + 1)^2 + M)$ storage.[20] The cost of each arithmetic operation depends on how many bits are needed to represent the input fractions. The checker compares the bounds and the allocations attaining them against an independent enumeration of finite allocations, including overlapping intervals and inconsistent labour bounds.

When allocations to several tasks or types of work share a person's time, use the joint formulation in Proposition 4.2. The graph above represents one person's allocation to one type of work on one task. Its numerical examples check that calculation. Applying these bounds to workplace labour measurements also requires empirical checks of their accuracy and coverage of the included work.

References

1. Casper Albers and Daniël Lakens, *When power analyses based on pilot data are biased: Inaccurate effect size estimators and follow-up bias*, Journal of Experimental Social Psychology **74** (2018), 187–195, https://doi.org/10.1016/j.jesp.2017.09.004.
2. Elizabeth S. Allman, Catherine Matias, and John A. Rhodes, *Identifiability of parameters in latent structure models with many observed variables*, The Annals of Statistics **37**

[19] Vertices represent time boundaries. A directed edge represents an upper bound on the change in cumulative labour between two vertices, and a path adds the weights of its edges. Thus "shortest" means the smallest total weight. A negative edge weight can encode a lower labour bound after its inequality is reversed. A negative cycle makes the constraints inconsistent because adding its inequalities would require zero to be less than or equal to a negative number.

[20] Big-O notation describes an upper bound on growth, ignoring fixed multiplicative constants. There are $K + 1$ time-boundary vertices and M supplied interval totals or bounds. The leading terms here are cubic in the vertex count for arithmetic operations and quadratic for storage.

(2009), no. 6A, 3099–3132, Section 6.1, Theorem 6. https://jarhodesuaf.github.io/papers/Latent.pdf.
3. Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic, *Prediction-powered inference*, Science **382** (2023), no. 6671, 669–674, Author manuscript: arXiv:2301.09633v4. https://doi.org/10.1126/science.adi6000.
4. Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein, *Measuring the impact of early-2025 AI on experienced open-source developer productivity*, 2025, arXiv:2507.09089v2. https://arxiv.org/abs/2507.09089v2.
5. Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud, *We are changing our developer productivity experiment design*, METR research update, February 2026, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/.
6. Stephen Boyd and Lieven Vandenberghe, *Convex optimization*, Cambridge University Press, 2004, Sections 5.2–5.3 and 5.6. https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf.
7. Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, June 2025, Author manuscript; Section 4.1 and Table 3. https://www.mertdemirer.com/Papers/Demirer_AI_productivity.pdf.
8. ———, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, Management Science (2026), Published online 27 February 2026. https://doi.org/10.1287/mnsc.2025.00535.
9. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction—CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, https://lean-lang.org/papers/lean4.pdf, pp. 625–635.
10. Nicholas Denis and Mohammed Haddou, *Prediction-powered estimation: Unbiased model-assisted estimation*, Journal of Official Statistics **42** (2026), no. 3, 371–394, https://doi.org/10.1177/0282423X261451320.
11. A. Dvoretzky, A. Wald, and J. Wolfowitz, *Relations among certain ranges of vector measures*, Pacific Journal of Mathematics **1** (1951), no. 1, 59–74, Theorem 4. https://msp.org/pjm/1951/1-1/pjm-v1-n1-p06-s.pdf.
12. Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler, *The SPACE of developer productivity: There's more to it than you think*, ACM Queue **19** (2021), no. 1, 20–48, https://doi.org/10.1145/3454122.3454124.
13. Martin Haugh, *An introduction to copulas*, IEOR E4602: Quantitative Risk Management, Columbia University, 2016, Spring 2016, Theorem 2. https://www.columbia.edu/~mh2078/QRM/Copulas.pdf.
14. D. G. Horvitz and D. J. Thompson, *A generalization of sampling without replacement from a finite universe*, Journal of the American Statistical Association **47** (1952), no. 260, 663–685, https://doi.org/10.1080/01621459.1952.10483446.
15. Michael G. Hudgens and M. Elizabeth Halloran, *Toward causal inference with interference*, Journal of the American Statistical Association **103** (2008), no. 482, 832–842, https://doi.org/10.1198/016214508000000292.
16. Kosuke Imai, Gary King, and Clayton Nall, *The essential role of pair matching in cluster-randomized experiments, with application to the mexican universal health insurance evaluation*, Statistical Science **24** (2009), no. 1, 29–53, https://doi.org/10.1214/08-STS274.
17. Piotr Indyk, *Lecture 17: Shortest paths IV: Applications*, 6.006 Introduction to Algorithms, Massachusetts Institute of Technology, 2011, Lecture slides, 7 April; difference constraints on slides 6–11. https://courses.csail.mit.edu/6.006/spring11/lectures/lec17.pdf.

18. M. Ali Khan, Kali P. Rath, and Yeneng Sun, *The Dvoretzky–Wald–Wolfowitz theorem and purification in atomless finite-action games*, International Journal of Game Theory **34** (2006), no. 1, 91–104, Section 2. https://doi.org/10.1007/s00182-005-0004-3.
19. Tim P. Morris, Ian R. White, and Michael J. Crowther, *Using simulation studies to evaluate statistical methods*, Statistics in Medicine **38** (2019), no. 11, 2074–2102, https://doi.org/10.1002/sim.8086.
20. Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz, *Reading between the lines: Modeling user behavior and costs in AI-assisted programming*, Proceedings of the CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, 2024, Author manuscript: arXiv:2210.14306v5. https://arxiv.org/abs/2210.14306v5, pp. 142:1–142:16.
21. Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer, *The impact of AI on developer productivity: Evidence from GitHub Copilot*, 2023, arXiv:2302.06590v1. https://arxiv.org/abs/2302.06590v1.
22. Gilda Piaggio, Diana R. Elbourne, Stuart J. Pocock, Stephen J. W. Evans, and Douglas G. Altman, *Reporting of noninferiority and equivalence randomized trials: Extension of the CONSORT 2010 statement*, JAMA **308** (2012), no. 24, 2594–2604, https://doi.org/10.1001/jama.2012.87802.
23. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, Association for Computing Machinery, 2020, https://leanprover-community.github.io/papers/mathlib-paper.pdf, pp. 367–381.
24. Alexander Torgovitsky, *Partial identification by extending subdistributions*, Quantitative Economics **10** (2019), no. 1, 105–144, https://doi.org/10.3982/QE634.
25. Ian Wright and Albert Ziegler, *The standard coder: a machine learning approach to measuring the effort required to produce source code change*, 2019, arXiv:1903.02436v1. https://arxiv.org/abs/1903.02436v1.